P0322r0 : exception_list

Project:	ISO JTC1/SC22/WG21: Programming Language C++
Number:	P0322r0
Date:	2016-04-24
Reply-to:	balelbach@lbl.gov
Author:	Bryce Adelstein Lelbach
Contact:	balelbach@lbl.gov
Author:	Alisdair Meredith
Contact:	alisdairm@me.com
Author:	Jared Hoberock
Contact:	jhoberock@nvidia.com
Audience:	Library Evolution Working Group (LEWG)
Audience:	Study Group 1 - Concurrency (SG1) for concurrency concerns and integration with the Parallelism TS
URL:	https://git.io/vrP2Q

1 Motivation

The Parallelism TS specifies `exception_list`, a class which owns a sequence of `exception_ptr` objects. `exception_list` is used to report exceptions that are thrown during the execution of a standard parallel algorithm. `exception_list` in the Parallelism TS specifies a query interface, but has no interface for constructing and populating the object.

At Jacksonville, there was interest in seeing the `exception_list` class from the Parallelism TS be elaborated into a more general-purpose and usable type. In particular, we want `exception_list` to have interfaces for construction which make it possible for standard library users to utilize exception_list in their own code.

2 Design

While exploring the design of `exception_list`, our major question was should `exception_list` be mutable or immutable after construction. The pros of both options are:

- Immutable
  - Existing exception types are immutable.
  - An immutable design negates many of our concerns regarding the use of `exception_list` in a multi-threaded context.
- Mutable
  - Existing standard library containers are mutable.
  - The standard library doesn't currently have a design for immutable containers and we will not have sufficient time before C++17 to full explore this design space.
  - A simple, non-concurrent mutable `exception_list` has decreased space and time overhead when compared to an immutable `exception_list`.

We decided upon an immutable design. The precedence for immutability in existing exception types was the major deciding factor. We did not wish to introduce a new standard exception type which had substantially different semantics from existing exception types.

Additionally, some of the authors had strong concerns about potential data races with `exception_list` which are alleviated by the immutable design. An immutable `exception_list` would be a persistent structure (e.g. older versions of the container would be preserved on "insertion" or "removal).

A reference-counted tree-like structure can be used to implement our immutable `exception_list`, similar to the implementation of immutable sequences via 2-3 finger trees in languages such as Haskell. Such structures allow for constant time access to and insertion at the ends of the sequence and logarithmic time splitting and concatenation.

If an immutable `exception_list` is shipped, we do not believe it will be possible to switch to a mutable design in the future. Such a switch would break code (at runtime) that was written assuming that the type was immutable.

A draft specification for an immutable `exception_list` with a constructor-based interface for concatenation. We have deleted the

move constructor and move assignment operator; these operations seem nonsensical for an immutable, persistent data structure where copying is cheap and moving would have to leave the moved-from object in an unchanged state (otherwise, it would no longer be truly immutable and thus would be susceptible to thread-safety issues).

Note that complexity guarantees in the below specification are based on a theoretical implementation using 2-3 finger trees.

3 Specification

```
namespace std {

class exception_list : public exception
{
  public:
    typedef /*** unspecified ***/ iterator;
    typedef /*** unspecified ***/ size_type;

    ////////////////////////////////////////////////////////////////////////
    // CONSTRUCTORS

    constexpr exception_list() noexcept = default;

    exception_list(const exception_list& other);
    exception_list& operator=(const exception_list& other);

    exception_list(exception_list&&) = delete
    exception_list& operator=(exception_list&&) = delete;

    // "push_back" constructors
    exception_list(exception_ptr e);
    exception_list(const exception_list& other, exception_ptr e);

    // iterator-pair "insert" constructors
    exception_list(iterator first, iterator last);
    exception_list(const exception_list& other,
                   iterator first, iterator last);
    template <class InputIterator>
    exception_list(InputIterator first, InputIterator last);
    template <class InputIterator>
    exception_list(const exception_list& other,
                   InputIterator first, InputIterator last);

    // initializer-list "insert" constructors
    exception_list(initializer_list<exception_ptr> list);
    exception_list(const exception_list& other,
                   initializer_list<exception_ptr> list);

    // "splice" constructor
    exception_list(const exception_list& other0,
                   const exception_list& other1) noexcept;

    ////////////////////////////////////////////////////////////////////////
    // QUERY INTERFACE

    size_type size() const noexcept;

    iterator begin() const noexcept;
    iterator cbegin() const noexcept;

    iterator end() const noexcept;
    iterator cend() const noexcept;

    ////////////////////////////////////////////////////////////////////////

    const char* what() const noexcept override;
};

}
```

The class `exception_list` owns a sequence of `exception_ptr` objects.

The type `exception_list::iterator` shall fulfill the requirements of `ForwardIterator`.

The type `exception_list::size_type` shall be an unsigned integral type large enough to represent the size of the sequence.

`constexpr exception_list() noexcept = default;`

Effect: Construct an empty `exception_list`.

```
exception_list(const exception_list& other);
```

Effect: Construct a new `exception_list` which is a copy of `other`.

Complexity: Linear time in the size of `other`.

Complexity: Constant time.

```
exception_list& operator=(const exception_list& other);
```

Effect: Copy the contents of `other` into this `exception_list`.

Complexity: Linear time in the size of `other`.

Complexity: Constant time.

```
exception_list(exception_ptr e);
```

Effect: Construct a new `exception_list` which contains a single element, `e`.

Complexity: Constant time.

```
exception_list(const exception_list& other, exception_ptr e);
```

Effect: Construct a new `exception_list` which is a copy of `other`, and append `e` to the end of the owned sequence.

Complexity: Linear in the size of `other` + 1.

Complexity: Constant time.

```
exception_list(iterator first, iterator last);
```

Effect: Construct a new `exception_list` which contains `distance(first, last)` elements from the range `[first, last)`.

Complexity: Logarthmic in `distance(first, last)`.

```
exception_list(const exception_list& other, iterator first, iterator last);
```

Effect: Construct a new `exception_list` which is a copy of `other`, and append the range `[first, last)` to the end of the owned sequence.

Complexity: Logarthmic in `min(other.size(), distance(first, last))`.

```
template <class InputIterator>

exception_list(InputIterator first, InputIterator last);
```

Effect: Construct a new `exception_list` which contains `distance(first, last)` elements from the range `[first, last)`.

Complexity: Linear in `distance(first, last)`.

Remarks: This constructor shall not participate in overload resolution if `is_convertible_v<InputIterator::value_type, exception_ptr> == false`.

```
template <class InputIterator>

exception_list(const exception_list& other, InputIterator first, InputIterator last);
```

Effect: Construct a new `exception_list` which is a copy of `other`, and append the range `[first, last)` to the end of the owned sequence.

Complexity: Linear in `distance(first, last)`.

Remarks: This constructor shall not participate in overload resolution if `is_convertible_v<InputIterator::value_type, exception_ptr> == false`.

```
exception_list(initializer_list<exception_ptr> list);
```

Effect: Construct a new `exception_list` which contains `list.size()` elements from `list`.

Complexity: Linear in the size of `list`.

```
exception_list(const exception_list& other, initializer_list<exception_ptr> list);
```

Effect: Construct a new `exception_list` which is a copy of `other`, and append `list` to the end of the owned sequence.

Complexity: Linear in the size of `list`.

```
exception_list(const exception_list& other0, const exception_list& other1);
```

Effect: Construct a new `exception_list` which contains all the elements of `other0` followed by all the elements of `other1`.

Complexity: Logarithmic in the `min(other0.size(), other1.size())`.

```
size_type size() const noexcept;
```

Returns: The number of `exception_ptr` objects contained within the `exception_list`.

Complexity: Constant time.

```
iterator begin() const noexcept;

iterator cbegin() const noexcept;
```

Returns: An iterator referring to the first `exception_ptr` object contained within the `exception_list`.

```
iterator end() const noexcept;

iterator cend() const noexcept;
```

Returns: An iterator that is past the end of the owned sequence.

```
const char* what() const noexcept override;
```

Returns: An implementation-defined NTBS.